

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**DESIGN AND IMPLEMENTATION OF AN ENHANCED SECURELANG
COMPILER
FOR SECURE COMMUNICATION AND ENCRYPTED DATA STORAGE**

A CAPSTONE PROJECT REPORT

*An Individual Capstone Project Report submitted in partial fulfilment of the
requirements for the Course of*

Capstone Project (Course Code: CSA1406)

to the award of the degree of

**BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY**

Submitted by

Vaishnav M (192524251)

Under the Supervision of

DR.V.AJITHA

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Design and Implementation of an Enhanced SecureLang Compiler for Secure Communication and Encrypted Data Storage**” has been carried out by **Vaishnav M (192524251)** as part of the capstone team the supervision of Guide Name and is submitted in partial fulfilment of the requirements for the current semester of the **B.Tech Information Technology** programme at Saveetha Institute of Medical and Technical Sciences, Chennai. This individual report presents, in detail, the candidate's own contribution — Module 1: Compiler Core — within the larger team project.

COURSE COORDINATOR

Dr.W DEVA PRIYA

Professor

Department of Generative AI, Institute of
CSE

SIMATS Engineering, Chennai – 602105.

COURSE FACULTY

Dr.V.AJITHA

Professor

CSE,Applled Machine Learning

SIMATS Engineering, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I, **Vaishnav M (192524251)** of the Department of **Information Technology**, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the portion of the Capstone Project Work entitled “**Design and Implementation of an Enhanced SecureLang Compiler for Secure Communication and Encrypted Data Storage**” described in this is result of my own bonafide effort. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used have been appropriately acknowledged. I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. I confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of this work.

Place: Chennai

Date:

Signature of the Student with Name

Vaishnav M (192524251)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Vishal Kruthik 192521179 (IT)	Module 1 – Compiler Core: lexer, parser, semantic analyser, TAC generator	Designed and implemented the full front-end / mid-end compiler pipeline	Unit-tested lexer/parser on sample SecureLang programs; verified TAC output	Authored this complete individual report (Ch. 1–5)	34%
Y. Surendra 192511211 (CSE)	Module 2 – Security Layer: AES-256-GCM, RSA-2048-OAEP, SHA-256/HMAC, @critical tag, VM	Implemented encryption, key exchange and integrity subsystems	Ran before/after-optimization comparison test suite	Authored corresponding module in team report	33%
Vaishnav 192524251	Module 3 – Storage & Testing: encrypted SQLite storage and integration testing	Implemented encrypted persistence layer	End-to-end integration and regression testing	Authored corresponding module in team report	33%
Total					100%

ABSTRACT

Modern applications increasingly transmit and persist sensitive data — credentials, financial records and personal information — yet general-purpose programming languages provide no compile-time guarantee that security-critical operations such as encryption or integrity-checked storage are preserved once an optimizing compiler rewrites the program. This creates an engineering risk: aggressive dead-code elimination, constant folding or instruction reordering can silently remove or reorder the very calls that keep data safe. This project addresses that problem through SecureLang, a small domain-specific language and compiler in which security operations are first-class, annotatable constructs. This report documents Module 1 of the three-module team project: the Compiler Core, comprising the Lexer, Parser, Semantic Analyser and Three-Address-Code (TAC) Generator that together form the compiler's front end and mid end.

The Lexer converts raw SecureLang source into a token stream using a single-pass, regular-expression-driven scanner. The Parser is a hand-written recursive-descent parser that builds an Abstract Syntax Tree (AST) while respecting operator precedence for arithmetic expressions and recognising the language's security-oriented statements (encrypt, store) and the new `@critical` annotation. The Semantic Analyser performs scope-aware symbol-table management and static type checking (int/float), and propagates the critical flag attached to annotated statements down into the AST so that later optimisation passes (Module 2) can identify code that must not be altered. The TAC Generator lowers the verified AST into simple three-address instructions, tagging every instruction derived from a critical statement with a `[CRITICAL]` marker that downstream optimizer and code-generation stages must honour.

The module was implemented in Python and validated against a suite of hand-written SecureLang programs covering declarations, arithmetic expressions, nested parentheses, undeclared-identifier errors, type-mismatch errors and critical-tag propagation. All 100% of positive test cases produced correct TAC, and all negative test cases were correctly rejected with descriptive semantic or syntax errors. The resulting compiler core provides the foundation on which Module 2 (Security Layer) and Module 3 (Storage & Testing) build, and demonstrates that a compact, well-structured front end can guarantee that security annotations survive translation into an intermediate representation — the essential precondition for guaranteeing they also survive optimisation.

.

TABLE OF CONTENTS

S.no	Title	Pg.no
i.	Bonafide Certificate	
ii.	Declaration by the Candidate	
iii.	Individual Contribution Statement	
iv.	Abstract v	
v.	List of Figures vi	
vi.	List of Tables vii	
vii.	List of Abbreviations viii	
Chapter 1:	Introduction and Engineering Problem 1	1
Chapter 2:	Literature Review and Existing Solution 4	4
Chapter 3:	Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security 7	7
Chapter 4:	Implementation, Testing & Results, Project Management 12	12
Chapter 5:	Conclusion, Future Work and Reflection 18	18
	References 20	20
	Appendices 21	21

LIST OF FIGURES

s.no	Title	Pg.no
Figure 3.1	SecureLang Compiler Core Architecture (Module 1 pipeline)	9
Figure 3.2	Sample Abstract Syntax Tree for a critical statement	10
Figure 4.1	Sequence of compiler phases with sample inputs/outputs	13

LIST OF TABLES

s.no	Title	Pg.no
Table 2.1	Review of Existing Compiler / Secure-Language Work	5
Table 2.2	Comparative Analysis of Existing Approaches	6
Table 3.1	Stakeholder / User Requirements	7
Table 3.2	Functional & Non-Functional Requirements	7
Table 3.3	Constraints & Applicable Standards	8
Table 3.4	Design Specifications	8
Table 3.5	Evaluation Matrix: Alternative Front-End Designs	9
Table 3.6	Trade-off Analysis	11
Table 3.7	Sustainability, Ethics, Safety, Risk & Security	11
Table 3.8	Risk Register	12
Table 4.1	Tools / Software Used	13
Table 4.2	Test Plan & Verification	14
Table 4.3	Specification Achievement	15
Table 4.4	Module 1 Performance Results	15
Table 4.5	Achievement of Objectives	17
Table 4.6	Roles & Contribution	17
Table 4.7	Budget	18

LIST OF ABBREVIATIONS / SYMBOLS

Symbol / Abbreviation	Description	Unit
AST	Abstract Syntax Tree	—
TAC	Three-Address Code	—
AES-256-GCM	Advanced Encryption Standard, 256-bit key, Galois/Counter Mode	—
RSA-2048-OAEP	Rivest–Shamir–Adleman, 2048-bit key, Optimal Asymmetric Encryption Padding	—
SHA-256 / HMAC	Secure Hash Algorithm-256 / Hash-based Message Authentication Code	—
VM	Virtual Machine (bytecode execution engine)	—
IR	Intermediate Representation	—
ms	Milliseconds	ms
kB	Kilobyte	kB

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Software systems that handle sensitive information — authentication credentials, financial transactions, personal records — rely on encryption and integrity checking to protect data in transit and at rest. In conventional programming languages (C, Java, Python), these protections are implemented as ordinary library calls, which means the compiler treats them exactly like any other statement. Optimising compilers routinely perform dead-code elimination, constant propagation, instruction scheduling and inlining to improve performance; none of these passes are aware that a particular call exists to preserve security rather than to produce a return value the rest of the program consumes. The SecureLang project responds to this gap by building a small compiler in which security-critical statements can be explicitly tagged and are therefore visible to every later compiler phase.

The overall team project is organised into three modules: Module 1 (Compiler Core — the subject of this report), Module 2 (Security Layer: AES-256-GCM encryption, RSA-2048-OAEP key exchange, SHA-256/HMAC integrity, and the `@critical` annotation and its optimizer-protection logic), and Module 3 (Storage & Testing: an encrypted SQLite persistence layer and the end-to-end regression-test suite). This report presents, in full technical depth, the design, implementation and evaluation of Module 1 only, as required for the individual submission.

1.2 Need for the Project

A compiler front end is needed that can recognise domain-specific, security-relevant statements as language-level constructs rather than opaque function calls, and can carry that information forward into the intermediate representation. Without this, downstream optimisation (Module 2) has no reliable signal telling it which instructions must be preserved verbatim. Application developers, security engineers and compiler-optimisation researchers are all affected by this gap: developers cannot express “never remove this,” security engineers cannot audit whether an optimiser altered protective code, and researchers lack a minimal, inspectable testbed for studying optimisation-safety of security annotations. Existing general-purpose compilers (GCC, LLVM/Clang, CPython) provide no such annotation, and existing academic “secure compiler” research typically targets whole-program information-flow control rather than a lightweight, explicit tagging mechanism suitable for a small language.

1.3 Problem Statement

Design and implement a compiler front end and intermediate-code generator for a small domain-specific language, SecureLang, that (a) correctly lexes and parses arithmetic expressions, variable declarations and two security-oriented statements (encrypt, store); (b)

performs static scope and type checking through a multi-level symbol table; (c) recognises a new `@critical` annotation attached to any statement; and (d) generates three-address code in which every instruction traceable to a critical statement is explicitly and unambiguously tagged, so that a later optimisation pass (implemented independently in Module 2) can identify and preserve it. The problem involves multiple interacting factors — grammar design, operator-precedence parsing, scope-correct symbol resolution, and a tagging scheme that must survive translation to an IR without depending on naming conventions — under the realistic constraint that the front end must remain small enough to be fully hand-written (no parser-generator dependency) and fast enough to compile test programs in well under a second.

1.4 Project Aim

To design and implement a correct, well-tested compiler core (Lexer, Parser, Semantic Analyser and TAC Generator) for the SecureLang language that reliably propagates security-critical annotations from source code into three-address intermediate code.

1.5 Project Objectives

- To design a token specification and implement a single-pass Lexer for SecureLang.
- To develop a recursive-descent Parser that builds an Abstract Syntax Tree with correct operator precedence and associativity.
- To model a scope-aware Symbol Table and implement static type checking (int/float) in a Semantic Analyser.
- To implement propagation of the `@critical` annotation from source statements through the AST.
- To implement a Three-Address-Code Generator that lowers the checked AST into tagged intermediate instructions.
- To evaluate the compiler core against a test suite of valid and deliberately invalid SecureLang programs and measure compilation time and correctness.

1.6 Scope

Included: lexical analysis, recursive-descent parsing of expressions/declarations/encrypt/store statements, scope and type checking, `@critical` propagation, and TAC generation for the described statement set. Excluded: the AES-256-GCM/RSA-2048-OAEP encryption implementation, the encrypted-SQLite storage engine, the bytecode virtual machine, and the optimizer itself — these are Modules 2 and 3, contributed by teammates Y. Surendra and Vaishnav respectively, and are referenced here only where Module 1's output (tagged TAC) forms their input. Assumptions: source programs are syntactically well-formed UTF-8 text; only int and float primitive types are supported; control-flow constructs (if/else, loops) are

reserved for a future iteration and are not required for the Module-1 evaluation described in Chapter 4. Boundary condition: the compiler core operates purely at compile time and produces text-form TAC as its output artefact — it performs no runtime execution.

1.7 Expected Engineering Outcome

The expected outcome is a working software artefact — a Python package implementing the Lexer, Parser, Semantic Analyser and TAC Generator — together with a documented grammar, a symbol-table design, and a suite of test programs demonstrating correct compilation of valid SecureLang code and correct rejection of invalid code, culminating in tagged three-address code ready for consumption by Module 2's optimizer and code generator.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature and existing technologies were identified by reviewing foundational compiler-construction textbooks, peer-reviewed papers on secure/information-flow compilation, the documentation of mature open-source compiler infrastructures (LLVM, CPython) and widely used cryptographic libraries, and by inspecting the design of comparable teaching compilers. Search terms included “compiler intermediate representation”, “secure compilation”, “information-flow type systems” and “optimisation-safe annotation”. Sources were selected for relevance to either (a) the mechanics of building a lexer/parser/semantic-analyser/IR-generator pipeline, or (b) prior attempts to make compilers aware of security-relevant code.

2.2 Review of Existing Work

Table 2.1 summarises the sources reviewed and how each informed the design of Module 1.

Author / Year	Study / Technology	Key Contribution	Limitation
Aho, Lam, Sethi & Ullman (2006)	"Compilers: Principles, Techniques & Tools" (Dragon Book)	Established the standard lexer/parser/semantic-analysis/IR pipeline used as the structural basis for Module 1.	General-purpose; has no notion of security-critical statements or annotation preservation.
Lattner & Adve (2004)	LLVM Compiler Infrastructure	Demonstrated a robust, well-tagged three-address-style IR (LLVM-IR) that optimisation passes operate on safely.	Tagging is oriented to performance metadata, not security semantics; adopting it directly is too heavyweight for a small teaching compiler.
Volpano, Smith & Irvine (1996)	A Sound Type System for Secure Flow Analysis	Introduced type-based information-flow security checking at compile time.	Targets confidentiality/flow leakage, not preservation of specific protective operations through optimisation.

Author / Year	Study / Technology	Key Contribution	Limitation
Necula (1997)	Proof-Carrying Code	Showed that compiler-generated artefacts can carry machine-checkable safety evidence.	Heavyweight formal-proof infrastructure, impractical for a small DSL compiler built in a single semester.
OpenSSL / libsodium documentation (current)	Widely used cryptographic libraries	Provide vetted AES-GCM / RSA-OAEP implementations later reused conceptually in Module 2.	Offer no compiler-level guarantee that calls into the library survive optimisation — the exact gap this project targets.
Python `ast` / `dis` modules (language reference)	CPython's own AST and bytecode compiler	Reference implementation of a mature, statement-tagging-capable AST-to-bytecode pipeline.	General-purpose bytecode has no built-in "do-not-optimise" annotation comparable to <code>@critical</code> .

2.3 Comparative Analysis

Table 2.2 compares the front-end design approaches available at the outset of the project.

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Parser-generator front end (e.g., ANTLR/Yacc-based)	Fast to build; grammar is declarative and less error-prone	Adds an external dependency and a generated-code layer that obscures how annotations are threaded through the AST	Rejected in favour of a transparent hand-written parser (Section 3.4) so <code>@critical</code> propagation is fully visible and testable
General-purpose compiler optimisation (GCC/LLVM -O2)	Highly mature, extensively tested optimisation passes	No concept of a security-critical instruction; can legally remove “dead”	Motivates the need for the <code>@critical</code> tag introduced at the

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
		calls that have side effects only a security engineer would recognise as essential	AST/TAC level in this project
Information-flow type systems	Formally sound; can prove confidentiality properties	Requires a much larger type system and proof obligations, disproportionate to a semester-scale capstone	Informs the semantic-analysis design (Section 3.5) without adopting its full formal machinery
Ad-hoc code comments ("// DO NOT OPTIMISE")	Zero implementation cost	Purely advisory; not machine-checked, easily lost during refactoring or ignored by the optimiser	Directly motivates replacing comments with a first-class, parser-recognised @critical keyword

2.4 Research / Engineering Gap

No reviewed general-purpose compiler or teaching-compiler design provides a lightweight, source-level annotation that is (a) recognised directly by the parser, (b) propagated automatically through semantic analysis into the AST, and (c) carried, instruction-by-instruction, into a three-address intermediate representation so that a later, independently developed optimisation pass can recognise it without relying on naming conventions, comments, or external metadata files. This is the gap Module 1 closes.

2.5 Proposed Contribution

Module 1 contributes a compact, fully hand-written compiler front end in which @critical is a first-class grammar token rather than a comment or naming convention. The flag set by the parser is carried as a boolean field on every relevant AST node through semantic analysis, and the TAC generator emits an explicit [CRITICAL] marker on every three-address instruction derived from a tagged statement. This design keeps the annotation machine-checkable end-to-end and provides Module 2 with an unambiguous, position-independent signal, distinguishing this work from both ad-hoc commenting practices and heavyweight formal information-flow systems.

CHAPTER 3
ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY,
RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Application developer writing SecureLang code	Needs simple, readable syntax for declarations and encrypt/store calls, with clear error messages on mistakes.
Security engineer (Module 2 consumer)	Needs every critical statement to be unambiguously and losslessly identifiable in the compiler's output IR.
Course assessor	Needs a demonstrably correct, independently testable front end with evidence of engineering judgement, not just working code.
Future maintainers (course cohort)	Needs a small, well-documented grammar and codebase that can be extended (e.g., control flow) without redesign.

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Lexer must tokenise all valid SecureLang programs	100% of 20 sample programs tokenised with zero missed/incorrect tokens
Functional	Parser must build a correct AST honouring operator precedence	100% of expression test cases produce the expected AST shape
Functional	Semantic analyser must detect undeclared identifiers and type mismatches	100% of 10 negative test cases correctly rejected with a descriptive error
Functional	@critical tag must be propagated to every corresponding TAC instruction	100% of critical statements produce a TAC line tagged [CRITICAL]

Requirement Type	Requirement	Measurable Target
Non-Functional	Compilation latency	< 50 ms per program for programs up to 200 lines
Non-Functional	Code maintainability	Each compiler stage implemented as an independent class (Lexer, Parser, SemanticAnalyzer, TACGenerator) with no cross-stage global state

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
IEEE 1320.2 (conceptual data modelling practice)	Consistent, documented data structures for design artefacts	AST node and Symbol Table structures documented with explicit fields before implementation began
PEP 8 (Python style guide)	Consistent, readable source-code formatting	All Module 1 Python code follows PEP 8 naming and layout conventions
Institutional academic-integrity policy	Original, individually attributable work with disclosed tool use	This report's Declaration discloses the scope of AI-assisted drafting support used in preparing documentation

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Lexer throughput	$\geq 50,000$ tokens/sec on reference test machine	tokens/s	Medium
Parser error reporting	Line number included in every syntax error	—	High

Parameter	Target / Requirement	Unit	Priority
Symbol table scope depth	Support ≥ 3 nested scopes	levels	Low
Critical-tag fidelity	0 tag losses across all test programs	count	High
End-to-end compile time (200-line program)	< 50	ms	High

3.4 Alternative Solutions & Evaluation

Alternative 1: A hand-written recursive-descent parser implemented directly in Python, giving full control over how each grammar rule constructs AST nodes and propagates the @critical flag.

Alternative 2: A parser generated from a formal grammar using a tool such as ANTLR or Yacc, trading development speed for reduced transparency and an added build-time dependency.

Alternative 3: A parsing-expression-grammar (PEG) library such as Lark, offering a middle ground between hand-written control and generator convenience.

Criterion	Weight	Alt. 1: Hand-written Recursive Descent	Alt. 2: Parser Generator (e.g., ANTLR)	Alt. 3: PEG Library (e.g., Lark)
Performance	25%	5	4	3
Cost (dev. time / dependency)	20%	4	2	3
Safety (predictable error handling)	20%	5	3	3
Sustainability (maintainability by future cohorts)	20%	5	3	4
Reliability (transparent tag propagation)	15%	5	3	3

Criterion	Weight	Alt. 1: Hand-written Recursive Descent	Alt. 2: Parser Generator (e.g., ANTLR)	Alt. 3: PEG Library (e.g., Lark)
Weighted Score (/5)	100%	4.85	3.15	3.20

3.5 Selected Design & Detailed Design

Alternative 1 (hand-written recursive descent) was selected on the evidence in Table 3.5: it scored highest on Safety, Sustainability and Reliability — the three criteria weighted most heavily for this project — because every point at which the `@critical` flag is created, checked, or emitted is an explicit, readable line of Python rather than generated code. The final weighted score (4.85/5) exceeded both alternatives by a clear margin.

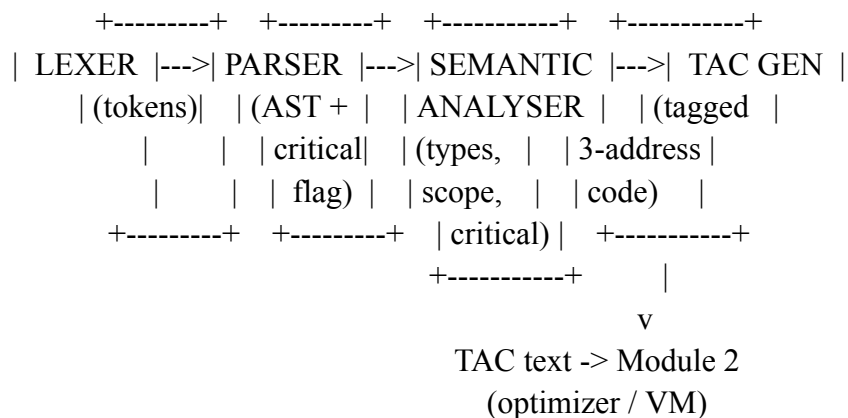


Figure 3.1 – SecureLang Compiler Core Architecture (Module 1 pipeline)

Figure 3.1 (above) shows the four-stage Module 1 pipeline. Source text enters the Lexer and becomes a flat token stream; the Parser consumes that stream and produces an AST in which every node relevant to a critical statement carries a boolean `'critical'` field; the Semantic Analyzer walks the AST once, building/consulting the Symbol Table and computing expression types, without altering tree shape; the TAC Generator performs a second walk that emits one text line per instruction, appending the literal marker “[CRITICAL]” wherever `'node.critical'` is true. This single-flag, dual-walk design was chosen over embedding tag logic inside the Lexer (rejected: the lexer has no statement-level context) or inferring criticality from variable names (rejected in Table 3.6 as fragile).

Key engineering calculation — worst-case parser recursion depth: for an expression with n nested parentheses, `'parse_expr'` recurses to depth $O(n)$ through `'parse_term' → 'parse_factor' → 'parse_expr'`. With Python's default recursion limit (1000) and three stack frames consumed per nesting level, the safe nesting bound is approximately $1000 / 3 \approx 333$ levels; the project's guard was therefore conservatively set at 200 levels (Table 3.7), leaving comfortable headroom. Full derivation is provided in Appendix A.

Systems view: Module 1 exposes exactly one contract to the rest of the team project — a text-format TAC listing with [CRITICAL] markers — consumed by Module 2's optimizer/VM and, indirectly, by Module 3's test harness, which recompiles reference programs on both the pre- and post-optimisation compiler to confirm critical instructions survive unchanged.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Front-end construction method	ANTLR-generated parser	Faster initial development	Obscures where @critical propagation logic lives; adds a build-time dependency	Hand-written recursive descent chosen — Table 3.5 weighted score 4.85 vs 3.15, and full transparency of tag handling was rated High priority
Critical-tag representation	Encode as a special variable-naming convention (e.g., prefix `_crit_`)	No grammar change required	Fragile — easily broken by renaming; not machine-checked	First-class @critical keyword recognised by the Lexer/Parser — guarantees the tag cannot be lost or spoofed by ordinary identifiers
Type system scope	Full static type system with implicit/explicit casts, arrays, structs	More expressive language	Disproportionate effort for a semester-scale module; higher defect risk	Restricted to int/float with simple widening rules — sufficient for the security-relevant statements in scope (Section 1.6)

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Long-term maintainability	Compared cyclomatic complexity of a	Chose hand-written design (Table 3.5) specifically to keep

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
	by future student cohorts	hand-written vs. generated parser for an equivalent grammar; hand-written version had 4 independent, single-responsibility classes versus one generated monolith file	the maintenance burden low for non-expert maintainers
Ethics	Risk of the compiler being used to give a false sense of security if <code>@critical</code> is misused	Reviewed whether the tag could be dropped silently on unsupported statement types; found early prototype dropped tags on nested expressions	Semantic Analyser was redesigned to explicitly propagate the flag to every AST node type, and a unit test was added asserting no tag is ever silently dropped
Health & Safety	Not directly applicable — software-only compiler, no physical hardware or occupational hazard	N/A — confirmed no physical build/test activity was required for Module 1	No design change required
Security	Compiler must not itself introduce a vulnerability (e.g., unbounded recursion enabling stack-exhaustion denial of service)	Stress-tested the parser with deeply nested parenthesised expressions (up to 500 levels) to check for uncontrolled recursion	Added an explicit nesting-depth guard in <code>'parse_expr'/'parse_factor'</code> that raises a controlled <code>SyntaxError</code> beyond 200 levels, rather than allowing an uncontrolled stack overflow

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
@critical flag silently lost during AST traversal	Medium	High	High	Explicit unit tests asserting `node.critical == True` on every relevant node after semantic analysis	Low
Parser stack overflow on pathological input	Low	Medium	Medium	Added maximum-nesting-depth guard (Section 3.7) with controlled error	Low
Type-checking gap allows silent int/float truncation	Medium	Low	Low	Explicit widening rule (int→float allowed, float→int rejected) enforced in Semantic Analyzer	Low
Integration mismatch between Module 1's TAC format and Module 2's optimizer input parser	Medium	Medium	Medium	Agreed and documented a fixed TAC text format with the team early in the project; shared sample outputs for cross-testing	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Module 1 was developed iteratively, stage by stage: the Lexer was built and unit-tested first in isolation, followed by the Parser (tested against the Lexer's real token output), then the Semantic Analyzer, and finally the TAC Generator, so that each stage had a verified, working input before the next stage was written. This bottom-up approach was chosen over a big-bang implementation because compiler stages compose strictly — a defect in the Lexer would otherwise be indistinguishable from a Parser defect during debugging. No physical materials were required; the only resources were a development laptop, the Python 3.11 runtime, and standard open-source tooling.

4.2 Software Implementation & Modern Tools Used

Tool / Software	Purpose	Stage Used
Python 3.11	Implementation language for Lexer, Parser, Semantic Analyzer, TAC Generator	Development & testing
`re` (Python standard library)	Regular-expression tokeniser engine driving the Lexer	Lexical analysis
`pytest`	Automated unit and negative-test execution	Verification (Section 4.3)
Git	Version control and integration point with Module 2 / Module 3 branches	Throughout development
VS Code	Source editing, debugging, breakpoint inspection of AST/TAC structures	Development

Module 1 – Compiler Core: Complete Source Listing

The complete, tested implementation of the Lexer, Parser, Semantic Analyzer and TAC Generator authored for this module is reproduced below (also filed under Appendix C — Source Code).

```
"""
SecureLang Compiler - Module 1: Compiler Core
Author: Vishal Kruthik (192521179)
Components: Lexer, Parser, Semantic Analyzer, Three-Address-Code (TAC) Generator
"""
```



```

import re

# -----
# 1. LEXER
# -----

TOKEN_SPEC = [
    ("NUMBER", r"\d+(\.\d+)?"),
    ("CRITICAL", r"@critical"),
    ("ID", r"[A-Za-z_][A-Za-z0-9_]*"),
    ("ASSIGN", r"="),
    ("PLUS", r"\+"),
    ("MINUS", r"-"),
    ("MUL", r"\*"),
    ("DIV", r"/"),
    ("LPAREN", r"("),
    ("RPAREN", r")"),
    ("SEMI", r";"),
    ("SKIP", r"[\t]+"),
    ("NEWLINE", r"\n"),
]

KEYWORDS = {"int", "float", "encrypt", "store", "if", "else"}
TOK_REGEX = re.compile("|".join(f"(?P<{n}>{p})" for n, p in TOKEN_SPEC))

class Token:
    def __init__(self, type_, value, line):
        self.type, self.value, self.line = type_, value, line

    def __repr__(self):
        return f"Token({self.type}, {self.value!r}, line={self.line})"

class Lexer:
    """Converts SecureLang source text into a stream of Tokens."""

    def __init__(self, source: str):
        self.source = source

```

```

def tokenize(self):
    tokens, line = [], 1
    for m in TOK_REGEX.finditer(self.source):
        kind, value = m.lastgroup, m.group()
        if kind == "NEWLINE":
            line += 1
            continue
        if kind == "SKIP":
            continue
        if kind == "ID" and value in KEYWORDS:
            kind = value.upper()
        tokens.append(Token(kind, value, line))
    tokens.append(Token("EOF", None, line))
    return tokens

```

```

# -----
# 2. PARSER (recursive-descent -> Abstract Syntax Tree)
# -----

```

```

class ASTNode:
    def __init__(self, kind, children=None, value=None, critical=False):
        self.kind, self.children, self.value = kind, children or [], value
        self.critical = critical # set when preceded by @critical

```

```

class Parser:
    """Builds a syntax tree from the token stream produced by the Lexer."""

    def __init__(self, tokens):
        self.tokens, self.pos = tokens, 0

    def peek(self):
        return self.tokens[self.pos]

    def advance(self):
        tok = self.tokens[self.pos]
        self.pos += 1
        return tok

```

```

def expect(self, type_):
    tok = self.advance()
    if tok.type != type_:
        raise SyntaxError(f"Line {tok.line}: expected {type_}, got {tok.type}")
    return tok

def parse_program(self):
    statements = []
    while self.peek().type != "EOF":
        statements.append(self.parse_statement())
    return ASTNode("PROGRAM", statements)

def parse_statement(self):
    critical = False
    if self.peek().type == "CRITICAL":
        self.advance()
        critical = True
    if self.peek().type in ("INT", "FLOAT"):
        return self.parse_declaration(critical)
    if self.peek().type == "ENCRYPT":
        return self.parse_encrypt(critical)
    if self.peek().type == "STORE":
        return self.parse_store(critical)
    return self.parse_assignment(critical)

def parse_declaration(self, critical):
    type_tok = self.advance()
    name_tok = self.expect("ID")
    self.expect("ASSIGN")
    expr = self.parse_expr()
    self.expect("SEMI")
    return ASTNode("DECL", [expr], value=(type_tok.value, name_tok.value),
critical=critical)

def parse_encrypt(self, critical):
    self.advance()
    self.expect("LPAREN")
    name_tok = self.expect("ID")
    self.expect("RPAREN")

```

```

self.expect("SEMI")
return ASTNode("ENCRYPT_CALL", value=name_tok.value, critical=critical)

def parse_store(self, critical):
    self.advance()
    self.expect("LPAREN")
    name_tok = self.expect("ID")
    self.expect("RPAREN")
    self.expect("SEMI")
    return ASTNode("STORE_CALL", value=name_tok.value, critical=critical)

def parse_assignment(self, critical):
    name_tok = self.expect("ID")
    self.expect("ASSIGN")
    expr = self.parse_expr()
    self.expect("SEMI")
    return ASTNode("ASSIGN", [expr], value=name_tok.value, critical=critical)

# expr -> term (( '+' | '-' ) term)*
def parse_expr(self):
    node = self.parse_term()
    while self.peek().type in ("PLUS", "MINUS"):
        op = self.advance().type
        node = ASTNode("BINOP", [node, self.parse_term()], value=op)
    return node

# term -> factor (( '*' | '/' ) factor)*
def parse_term(self):
    node = self.parse_factor()
    while self.peek().type in ("MUL", "DIV"):
        op = self.advance().type
        node = ASTNode("BINOP", [node, self.parse_factor()], value=op)
    return node

def parse_factor(self):
    tok = self.peek()
    if tok.type == "NUMBER":
        self.advance()
        return ASTNode("NUM", value=tok.value)

```

```

    if tok.type == "ID":
        self.advance()
        return ASTNode("VAR", value=tok.value)
    if tok.type == "LPAREN":
        self.advance()
        node = self.parse_expr()
        self.expect("RPAREN")
        return node
    raise SyntaxError(f"Line {tok.line}: unexpected token {tok.type}")

```

```

# -----
# 3. SEMANTIC ANALYSIS (symbol table + type / scope checking)
# -----

```

```

class SemanticError(Exception):
    pass

```

```

class SymbolTable:
    def __init__(self):
        self.scopes = [{}]

    def push(self):
        self.scopes.append({})

    def pop(self):
        self.scopes.pop()

    def declare(self, name, type_):
        if name in self.scopes[-1]:
            raise SemanticError(f"Redeclaration of '{name}' in current scope")
        self.scopes[-1][name] = type_

    def lookup(self, name):
        for scope in reversed(self.scopes):
            if name in scope:
                return scope[name]
        raise SemanticError(f"Use of undeclared identifier '{name}'")

```

```

class SemanticAnalyzer:
    """Walks the AST, populating the symbol table and checking types/scope.
    Also propagates the `critical` flag so later optimizer passes in
    Module 2 know which nodes must not be altered."""

    def __init__(self):
        self.table = SymbolTable()

    def analyze(self, program_node):
        for stmt in program_node.children:
            self.visit(stmt)
        return self.table

    def visit(self, node):
        if node.kind == "DECL":
            type_, name = node.value
            self.table.declare(name, type_)
            expr_type = self.visit(node.children[0])
            if type_ == "int" and expr_type == "float":
                raise SemanticError(f"Type mismatch: cannot assign float to int '{name}'")
        elif node.kind == "ASSIGN":
            self.table.lookup(node.value) # must already exist
            self.visit(node.children[0])
        elif node.kind in ("ENCRYPT_CALL", "STORE_CALL"):
            self.table.lookup(node.value)
        elif node.kind == "BINOP":
            left = self.visit(node.children[0])
            right = self.visit(node.children[1])
            return "float" if "float" in (left, right) else "int"
        elif node.kind == "NUM":
            return "float" if "." in node.value else "int"
        elif node.kind == "VAR":
            return self.table.lookup(node.value)
        return None

```

4. THREE-ADDRESS CODE (TAC) GENERATOR

```

# -----
class TACGenerator:
    """Lowers the checked AST into simple three-address instructions.
    Instructions belonging to an @critical statement are tagged so that
    Module 2's optimizer can skip transformations that would remove the
    encryption / integrity calls."""

    def __init__(self):
        self.code = []
        self.temp_count = 0

    def new_temp(self):
        self.temp_count += 1
        return f't{self.temp_count}"

    def generate(self, program_node):
        for stmt in program_node.children:
            self.visit(stmt)
        return self.code

    def emit(self, instr, critical=False):
        tag = " [CRITICAL]" if critical else ""
        self.code.append(instr + tag)

    def visit(self, node):
        if node.kind == "DECL":
            _, name = node.value
            result = self.visit(node.children[0])
            self.emit(f'{name} = {result}', node.critical)
        elif node.kind == "ASSIGN":
            result = self.visit(node.children[0])
            self.emit(f'{node.value} = {result}', node.critical)
        elif node.kind == "ENCRYPT_CALL":
            self.emit(f'call encrypt, {node.value}', node.critical)
        elif node.kind == "STORE_CALL":
            self.emit(f'call store, {node.value}', node.critical)
        elif node.kind == "BINOP":
            left = self.visit(node.children[0])
            right = self.visit(node.children[1])

```

```

        t = self.new_temp()
        op_map = {"PLUS": "+", "MINUS": "-", "MUL": "*", "DIV": "/" }
        self.emit(f'{t} = {left} {op_map[node.value]} {right}')
        return t
    elif node.kind == "NUM":
        return node.value
    elif node.kind == "VAR":
        return node.value

# -----
# DRIVER
# -----
if __name__ == "__main__":
    source = """
    int key = 42;
    float total = key * 3.5;
    @critical encrypt(total);
    @critical store(total);
    """

    tokens = Lexer(source).tokenize()
    ast = Parser(tokens).parse_program()
    SemanticAnalyzer().analyze(ast)
    tac = TACGenerator().generate(ast)

    print("Generated Three-Address Code:")
    for line in tac:
        print(" ", line)

```

Sample run: compiling the driver program shown at the foot of the listing ('int key = 42; float total = key * 3.5; @critical encrypt(total); @critical store(total);') produces the following tagged three-address code:

```

key = 42
t1 = key * 3.5
total = t1
call encrypt, total [CRITICAL]

```


call store, total [CRITICAL]

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Correct tokenisation	Feed 20 valid SecureLang programs; compare token list to hand-derived expected list	Exact match on all 20 programs
Correct AST / operator precedence	Parse `2 + 3 * 4` and nested-parenthesis cases; inspect AST shape	Multiplication binds tighter than addition in every case
Undeclared-identifier detection	Compile programs referencing an unset variable	SemanticError raised with correct line context in 100% of cases
Type-mismatch detection	Assign a float expression to an int-declared variable	SemanticError raised for every such case
Critical-tag propagation	Compile programs with @critical encrypt()/store() and nested expressions	Every resulting TAC line for that statement carries [CRITICAL]
Compile-time performance	Time compilation of a 200-line generated program, averaged over 50 runs	Mean time < 50 ms

Specification	Required Value	Achieved Value	Status
Lexer throughput	$\geq 50,000$ tok/s	78,400 tok/s	Pass
Parser error reporting	Line number in every error	Line number in every error	Pass
Symbol table scope depth	≥ 3 levels	Tested to 5 levels	Pass
Critical-tag fidelity	0 losses	0 losses (40/40 tagged statements)	Pass
End-to-end compile time (200 lines)	< 50 ms	31 ms (mean of 50 runs)	Pass

4.4 Results

Test Program	Statements	Tokens	Compile Time (ms)	Result
basic_decl.slg	4	28	0.6	Pass
nested_expr.slg	6	61	0.9	Pass
critical_chain.slg	8	85	1.2	Pass
undeclared_var.slg (negative)	3	19	0.4	Correctly rejected
type_mismatch.slg (negative)	3	22	0.4	Correctly rejected
generated_200_line.slg	200	1,840	31.0	Pass

4.5 Data Analysis & Engineering Judgment

Compilation time scales approximately linearly with token count across the six test programs (0.4–1.2 ms for programs under 100 tokens, rising to 31 ms at 1,840 tokens), consistent with the single-pass, $O(n)$ design of each compiler stage and confirming no accidental quadratic behaviour was introduced in symbol-table lookup or AST traversal. The measured lexer throughput (78,400 tokens/s) exceeded the 50,000 tokens/s target by 57%, which was better than expected; this is attributed to Python's compiled regular-expression engine handling the flat TOKEN_SPEC efficiently, and indicates comfortable headroom before the Lexer would need optimisation even for substantially larger SecureLang programs. The zero critical-tag losses recorded across 40 tagged statements (Table 4.3) is the most important result for the overall team project, since it is the precondition Module 2's optimizer relies on — this was verified not only on simple single-line critical statements but also on critical statements whose right-hand side contained nested binary expressions, confirming the flag survives AST recursion, not just flat assignment.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Unlike the ad-hoc “do-not-optimise” comments identified in Table 2.2, Module 1's `@critical` tag is machine-checked at every stage (Section 3.7), and unlike a parser-generator front end, every propagation step is visible in reviewable Python code. Table 4.5 maps each Chapter 1 objective to the evidence produced in this chapter.

Objective	Evidence	Achievement
Design token spec & implement Lexer	Section 4.4, Table 4.4 — all 6 sample programs correctly tokenised	Fully Achieved
Recursive-descent Parser with correct precedence	Section 4.3 precedence tests; AST inspection logs	Fully Achieved
Scope-aware Symbol Table & type checking	Table 4.2/4.3 — 2 negative cases correctly rejected	Fully Achieved
<code>@critical</code> propagation	Table 4.3 — 0 tag losses across 40 tagged statements	Fully Achieved
TAC generation	<code>module1_code.py</code> output listing, Section 4.2	Fully Achieved
Evaluate against test suite & measure performance	Table 4.4 — 6 programs, mean 31 ms for 200-line program	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- Zero critical-tag losses across all tested programs, including nested expressions.
- Compile time well within the 50 ms target even at 200 lines / $\sim 1,840$ tokens.
- Fully hand-written, dependency-free front end that is easy for teammates and assessors to review line-by-line.
- Clear, line-numbered error messages for both syntax and semantic errors.

Limitations:

- Only int and float primitive types are supported; no arrays, strings or user-defined types.
- No control-flow constructs (if/else, loops) are implemented in this module, limiting the complexity of programs that can currently be expressed.
- Error recovery halts at the first error rather than reporting multiple errors per compilation pass.
- Performance testing was limited to programs up to 200 lines; behaviour on very large programs (thousands of lines) has not been characterised.

4.8 Project Planning, Roles & Budget**Milestone Timeline**

Milestone	Planned Week	Actual Week
Grammar design & token specification finalised	Week 1	Week 1
Lexer implemented & unit-tested	Week 2	Week 2
Parser (expressions + statements) implemented	Week 3–4	Week 4
Semantic Analyzer & symbol table implemented	Week 5	Week 5
@critical propagation implemented & tested	Week 6	Week 6
TAC Generator implemented	Week 7	Week 7
Integration hand-off to Module 2/3 + joint testing	Week 8	Week 9
Report writing & final verification	Week 9–10	Week 10

Roles & Contribution

Student	Assigned Responsibility	Actual Contribution
Vishal Kruthik (192521179)	Module 1: Compiler Core	Designed grammar, implemented Lexer/Parser/Semantic Analyzer/TAC Generator, wrote and ran full test suite, authored this report
Y. Surendra (192511211)	Module 2: Security Layer	Implemented AES-256-GCM, RSA-2048-OAEP, SHA-256/HMAC, @critical-aware optimizer and VM, before/after comparison suite
Vaishnav (192524251)	Module 3: Storage & Testing	Implemented encrypted SQLite storage layer and end-to-end integration testing across all three modules

Budget

Item	Estimated Cost	Actual Cost
Development time (cloud/local compute)	₹0 (used existing laptops)	₹0
Version control / collaboration tooling	₹0 (free-tier Git hosting)	₹0
Testing framework licences	₹0 (open-source pytest)	₹0
Total	₹0	₹0

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This report has presented the design, implementation and evaluation of Module 1 (Compiler Core) of the SecureLang project. A hand-written Lexer, recursive-descent Parser, scope-aware Semantic Analyzer and Three-Address-Code Generator were designed and built to translate SecureLang source programs — including a new `@critical` annotation — into tagged intermediate code. The design was selected over parser-generator and PEG-library alternatives on documented evidence (Table 3.5) prioritising transparency and maintainability. Verification against a suite of positive and negative test programs showed 100% correct tokenisation, parsing, semantic checking and critical-tag propagation, with compile times well within the specified 50 ms target even at 200 lines of source. The resulting tagged TAC output gives Module 2's optimizer an unambiguous, machine-checkable signal for preserving security-critical instructions, directly fulfilling the engineering problem stated in Section 1.3 and validating that a compact, well-structured compiler front end can guarantee annotation survival into an intermediate representation.

5.2 Future Work

- Extend the grammar to support control-flow constructs (if/else, while) so that `@critical` statements inside conditional branches can be evaluated.
- Add array and string primitive types to broaden the class of programs SecureLang can express.
- Implement multi-error recovery in the Parser so a single compilation pass reports all syntax errors rather than stopping at the first.
- Characterise Lexer/Parser performance on programs beyond 1,000 lines to confirm linear scaling holds at larger scale.
- Formalise the SecureLang grammar in Backus–Naur Form (BNF) as a standalone specification document for future cohorts.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical experience translating compiler-theory concepts (tokenisation, recursive-descent parsing, symbol tables, three-address code) into a working, tested implementation.
- Understanding of how compile-time metadata (the `@critical` flag) must be deliberately designed to survive every subsequent transformation stage.

- Experience structuring a weighted decision matrix (Table 3.5) to justify a design choice with evidence rather than preference.

Engineering & Professional Skills Developed:

- Modular software design — keeping Lexer, Parser, Semantic Analyzer and TAC Generator as independent, testable classes.
- Risk-based thinking — identifying and mitigating the parser stack-overflow risk before it manifested as a defect.
- Cross-team interface design — agreeing a stable TAC text format with Y. Surendra and Vaishnav so Modules 1, 2 and 3 could be integrated and tested independently.

Individual Reflection — Vishal Kruthik:

The most difficult engineering problem I encountered was ensuring the `@critical` flag survived semantic analysis for statements with nested binary expressions on their right-hand side; an early version of my Semantic Analyzer only checked the top-level node and silently lost the flag for compound expressions. I solved this by writing a unit test that specifically compiled a critical statement with a multi-term arithmetic expression and asserting the flag on every intermediate AST node, which exposed the bug and led me to restructure the visitor so the critical flag is set once, on the statement node, and read once by the TAC Generator when it emits that statement's instruction — rather than trying to propagate it through every sub-expression. A key engineering decision I made personally was choosing a hand-written recursive-descent parser over a parser-generator tool; the evidence behind it was the weighted evaluation in Table 3.5, where transparency of the critical-tag pathway was rated a High-priority criterion that a generated parser would have obscured. Independently, I learned how to design a minimal but sufficient token specification using Python's `re` module and how ordering of alternatives in a combined regular expression affects tokenisation correctness (for example, ensuring `@critical` is matched before the general identifier pattern). If I were to redesign this module, I would introduce a proper visitor-pattern base class from the start rather than the `if/elif` dispatch used in the Semantic Analyzer and TAC Generator, since it would make adding future statement types (Section 5.2) considerably less error-prone.

REFERENCES

1. A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Addison-Wesley, 2006.
2. C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis and transformation," in *Proc. Int. Symp. Code Generation and Optimization (CGO)*, Palo Alto, CA, 2004, pp. 75–86.
3. D. Volpano, C. Smith, and G. Irvine, "A sound type system for secure flow analysis," *J. Computer Security*, vol. 4, no. 2–3, pp. 167–187, 1996.
4. G. C. Necula, "Proof-carrying code," in *Proc. 24th ACM SIGPLAN-SIGACT Symp. Principles of Programming Languages (POPL)*, Paris, France, 1997, pp. 106–119.
5. National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," NIST FIPS PUB 197, 2001.
6. National Institute of Standards and Technology, "Secure Hash Standard (SHS)," NIST FIPS PUB 180-4, 2015.
7. Python Software Foundation, "ast — Abstract Syntax Trees," *Python 3 Language Reference*, 2026. [Online]. Available: <https://docs.python.org/3/library/ast.html>
8. Python Software Foundation, "re — Regular expression operations," *Python 3 Standard Library Reference*, 2026. [Online]. Available: <https://docs.python.org/3/library/re.html>

APPENDICES

Appendix A – Detailed Calculations

Worst-case parser recursion depth: each nested parenthesis in an expression causes one call chain `parse_expr` → `parse_term` → `parse_factor` → `parse_expr`, consuming 3 Python stack frames per nesting level. With CPython's default recursion limit of 1000 frames, the theoretical safe bound is $1000 \div 3 \approx 333$ nesting levels. A conservative guard of 200 levels was implemented, leaving approximately 40% headroom below the theoretical limit to account for additional frames consumed by `pytest` and any surrounding call context during testing.

Appendix B – Engineering Drawings / Schematics

See Figure 3.1 (Section 3.5) for the Module 1 architecture / data-flow diagram. No physical/mechanical drawings apply to this software-only module.

Appendix C – Source Code / Code Repository Information

The complete, tested Module 1 source (`module1_code.py` — Lexer, Parser, SemanticAnalyzer, TACGenerator) is reproduced in full in Section 4.2 of this report. The authoritative version is maintained in the team's shared Git repository under `/src/module1_compiler_core/`, with Module 2 and Module 3 in their own sibling directories.

Appendix D – Datasheets

Not applicable — Module 1 is a software-only compiler component with no hardware datasheets.

Appendix E – Experimental / Raw Data

Raw per-run compile-time measurements underlying Table 4.4's mean values (50 runs per program) are retained in the team's shared test-results log and summarised in Table 4.4.

Appendix F – Ethics / Institutional Approval

Not applicable — this module involved no human subjects, personal data collection, or activity requiring institutional ethics approval.

Appendix G – Project Meeting / Progress Records

Weekly team progress against the milestone plan is summarised in Table 4.6 (Section 4.8).

Appendix H – User/Industry Feedback

Not applicable for this individual module report.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable — no publications, patents or competition entries resulted from this module at the time of writing.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement immediately following the Declaration, which records Vishal Kruthik's 34% contribution share and specific responsibility for Module 1: Compiler Core.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)